0.1 Abstract

My first paper compared Firefox and Chrome security on Android and argued the choice isn't binary – it depends on what you're protecting against. GrapheneOS pushed back hard, and they were right about several things I got wrong. This paper fixes those mistakes.

I oversold `isolatedProcess` as "the strongest sandbox" when really it's the only sandboxing mechanism Android makes available to apps. I left out Chromium's V8 sandbox, Oilpan GC, CFI, and a bunch of other mitigations. Those omissions made the comparison unfair, and I've corrected them here.

But the core claim survived: these two engines optimize for fundamentally different threats. Chromium is a containment architecture – it assumes you'll get compromised and limits the blast radius. Firefox is a prevention architecture – it reduces the chance you get compromised in the first place, using Rust to eliminate entire bug classes at compile time. These aren't competing claims about which is "more secure." They're different answers to different questions.

I also found genuinely new stuff while digging deeper – things I didn't know when I wrote the first draft. The V8 sandbox is more interesting than I realized. The extensions breaking site isolation problem affects both browsers, not just one. And Firefox's lack of CFI in SpiderMonkey is a real gap I should have caught.

The tl;dr: I got some things wrong, fixed them, and the main argument is stronger for it.

0.2 1. Introduction

I wrote the first paper because the public debate around Firefox vs Chrome security was stuck. One side kept saying Firefox has no sandbox, which was always overstated. The other kept saying Firefox is more secure because of Rust, which ignores real gaps. I wanted to land somewhere in the middle: these are different architectures optimized for different risks, and calling either "categorically more secure" misses the point.

The response was immediate. GrapheneOS published a detailed rebuttal on Reddit, and their Discord community engaged extensively. Some of what they said was fair – I'd made real errors. Some of it felt less like technical correction and more like dismissal. Characterizing independent analysis as "dishonest" and "unethical" because it gets details wrong isn't how peer review works. But I'm separating those two things here.

The corrections are real and documented below. I was imprecise about `isolatedProcess`. I omitted important Chromium mitigations. I didn't catch the CFI gap in SpiderMonkey. Each of those is fixed.

The characterizations of intent I address separately – not because they matter as much, but because the dynamic they create (ad hominem attacks on independent researchers) is worth naming. When every mistake gets framed as bad faith, fewer people do independent research. That's bad for everyone.

0.3 2. What I Still Stand By

Not everything in the original paper was wrong. Actually, most of it held up. Here's what survived.

0.3.1 2.1 Firefox's Rust Bet Is Paying Off

Firefox has rewritten major subsystems in Rust: the CSS engine (Stylo), the renderer (WebRender), library sandboxing (RLBox), parts of the networking stack. Rust eliminates entire bug classes at compile time – use-after-free, buffer overflows, null pointer dereferences. Chromium has mitigations for these (covered below), but mitigations manage symptoms. Rust eliminates the root cause.

The numbers back this up. Roughly 70% of Chromium's critical-severity bugs are memory safety issues [14]. Firefox's Rust components have produced zero such CVEs since they shipped [15]. That's not luck – it's what the language guarantees.

This doesn't mean Firefox is "more secure." It means the two projects took different paths to the same goal. Firefox prevents bugs from existing in the first place. Chromium prevents bugs from being exploitable when they do exist. Both approaches work.

0.3.2 2.2 Content Blocking != Privacy Theater

GrapheneOS called extension-based content blocking "privacy theater" – equating it with antivirus-style enumeration of badness. That analogy doesn't hold.

Blocking a request at the network layer before the payload reaches your device is structurally different from scanning a file after it's already on disk. uBlock Origin stops exploits from being delivered. That's not detection. That's prevention.

The real limitation (filter lists can't block novel zero-days) is real but incomplete as a critique. Zero-day delivery in practice relies on known malicious infrastructure – compromised ad networks, C2 domains, exploit kit landing pages. Filter lists block those. The "enumeration is futile" argument assumes attackers can spin up fresh infrastructure for every target at zero cost. That's not how mass-market exploitation works.

0.3.3 2.3 Monoculture Risk Is Real

Chromium's market share on mobile is overwhelming. When the Chromium engine has a critical vuln, billions of devices are affected simultaneously. That concentration of value creates attacker incentives that don't exist for Firefox's minority codebase.

This isn't about Firefox being "more secure." It's about attacker economics. A Chromium zero-day is worth more to an exploit broker than a Firefox zero-day, purely because of market share. Firefox being ignored by attackers is itself a security property, even if it's not a flattering one.

The dual-engine criticism ("Firefox adds a second engine to your device") is an Android platform constraint, not a Firefox deficiency. Android mandates a system WebView, and on GrapheneOS that WebView is Vanadium regardless of your browser choice. The decision to also run Firefox is a marginal attack surface increase weighed against monoculture risk reduction. Different users will weigh that differently.

0.3.4 2.4 Pick Your Threat Model

The original paper's core claim – browser choice is a threat-model alignment, not a binary security judgment – held up completely.

- If you're worried about targeted, state-sponsored attacks where the attacker has resources to find and weaponize a novel exploit, you want Chrome/Chromium. Post-compromise containment is your priority, and `isolatedProcess` delivers that.
- If you're worried about mass surveillance, ad-tech profiling, and drive-by exploits targeting the Chromium monoculture, Firefox makes more sense. A renderer that's harder to compromise in the first place reduces the probability that any exploit succeeds.
- If your primary concern is privacy (tracking, fingerprinting), Firefox's Total Cookie Protection, ETP, and container isolation are genuinely ahead of anything in the Chromium ecosystem.

These aren't competing claims about which browser is "more secure." They're different tools for different jobs.

0.3.5 2.5 Android vs. Desktop Matters

This is the point I keep coming back to. The sandbox gap everyone's arguing about is almost entirely Android-specific.

On Windows, Firefox's sandbox is at parity with Chrome's: - Both use AppContainer for kernel-level process isolation [13]. - Both restrict Win32k syscalls from renderer processes [9]. - Both use broker process models for privileged operations.

The `isolatedProcess` thing is an Android limitation because Android apps can't create namespaces or use seccomp-bpf – the Android Runtime needs too many syscalls. The `isolatedProcess` flag is the only sandboxing game in town on Android, and Firefox doesn't use it. That's real.

But Firefox's critics treat this as a universal Firefox deficiency when it's really a platform-specific gap. If the claim were "Firefox is categorically less secure than Chromium," you'd expect the sandbox gap to exist everywhere. On Windows, it doesn't. On Android, it does – and it must be weighed against Firefox's advantages (Rust, smaller API surface) that apply on all platforms.

Picking Chrome on Android because you prioritize mobile security is perfectly rational. Claiming "Firefox has no sandbox" or "Firefox is much more vulnerable" without qualifying the platform is overreach.

0.4 3. What I Got Wrong

Post-publication review caught several things. Here's what they were and how I fixed them.

0.4.1 3.1 I Left Out Half of Chromium's Mitigations

My first paper spent a lot of time on Firefox's Rust adoption and barely mentioned Chromium's mitigations. That made the comparison look one-sided even though the actual security picture is more balanced. Here's what I missed:

Mitigation	What It Does
V8 Sandbox	Locks JIT code to a reserved memory region so a JS exploit can't touch anything outside it
Oilpan GC	Garbage collector for DOM objects that eliminates use-after-free in rendering code
Mojo IPC	Type-checked message passing between processes so sandbox-escape exploits have fewer holes
PartitionAlloc	Hardened heap allocator with partition isolation, freelist entropy, MiraclePtr
Type-based CFI	Validates indirect function calls at runtime to block control-flow hijacking
MTE	Memory tagging at the hardware level (integrated into PartitionAlloc)

Chromium doesn't prevent memory bugs at the source the way Rust does, but it makes them significantly harder to exploit. That's a real investment, and ignoring it made my comparison incomplete.

0.4.2 3.2 The `isolatedProcess` Thing

I categorized GrapheneOS's "Firefox has no internal sandboxing" claim as "partially outdated." That was wrong. If you define sandboxing as kernel-level UID isolation (which is how GrapheneOS defines it), the claim is fully accurate. GeckoView doesn't use `isolatedProcess`. Full stop.

The disagreement isn't about facts. It's about definitions. GrapheneOS says sandboxing requires `isolatedProcess`. I was using a broader definition that includes process-level privilege separation. Both definitions are internally consistent. I should have been clearer about this from the beginning.

0.4.3 3.3 Fission Isn't Really Shipping on Android

My first paper mentioned Fission (Site Isolation) shipping on Android, then rolling back. But the abstract and conclusion referenced Fission like it was a current mitigation. It's not. Fission is disabled on release and beta channels as of Firefox 152. Nightly and developer builds have it at a reduced level (`ISOLATE_HIGH_VALUE` rather than full origin isolation). The paper should have been more careful about this.

0.4.4 3.4 Other Things I Missed

SpiderMonkey has no CFI. This is a real gap I should have caught. V8 has Clang's Cross-DSO Control Flow Integrity for indirect call validation [16]. SpiderMonkey doesn't. Since JS engines are the densest source of critical vulns in both browsers, this matters. Combined with V8's memory sandbox (which Firefox also lacks), Chromium's JS engine mitigation layer is genuinely stronger. Rust doesn't help here because JIT-generated code isn't subject to Rust's safety guarantees.

Hardware mitigations aren't automatic. I wrote that MTE, PAC, and BTI are "enforced at the OS and hardware level, not by the browser vendor." That's wrong. These features require allocator modifications, compiler support, and codebase validation. Chromium has done that work (MTE in PartitionAlloc, PAC/BTI validation). Firefox hasn't. This removes a false parity claim.

More CVEs doesn't mean less secure. My first paper's framing of Chromium's CVE count as evidence against it was sloppy. Chromium invests massively more in fuzzing, AI-guided auditing, and security research. Finding more bugs is a sign of more testing, not less security [18]. The relevant metric is residual risk after mitigation, not bug count – and that's not directly measurable from public data.

Extensions break site isolation in both browsers. I implied Firefox's extension model was more contained. It's not – extensions in both Firefox and Chromium run with cross-origin access that bypasses site isolation [19]. The real comparison isn't extensions vs no extensions. It's extensions vs built-in content filtering (as Brave does). That's a genuine trade-off with no clear winner, and my paper should have framed it that way.

0.5 4. Where We Still Disagree

Even after all the corrections above, some disagreements aren't about facts – they're about how you weigh them.

0.5.1 4.1 "Much More Vulnerable" vs "Differently Vulnerable"

GrapheneOS says Firefox is "much more vulnerable to exploitation." I don't think the evidence supports that, even after accounting for everything I got wrong.

Chrome has stronger post-compromise containment. Firefox has stronger pre-compromise prevention. These are different priorities, and calling one "much more vulnerable" treats containment as the only metric that matters. That's a value judgment, not a technical fact. My position is that both are rational depending on what you're protecting against, and I haven't seen evidence that changes that.

0.5.2 4.2 Advisory Latency

GrapheneOS's advisory on Firefox hasn't changed significantly since it was written, even as Firefox's architecture evolved. The Fission rollback is an example of this working both ways – I needed to correct my own framing, but the advisory also doesn't acknowledge that Firefox on Windows has

reached sandbox parity. Documentation latency is a real problem in security engineering [6], and it's not unique to any project.

0.6 5. The Personal Stuff

GrapheneOS called my paper "dishonest" and "unethical" in their Discord. On Reddit, they said it wasn't "based on real research or factual information" and that I was driven by "biases."

I'm separating this from the technical corrections because it's a different category of thing. Getting facts wrong isn't the same as being dishonest. An error means peer review worked. Calling someone dishonest for making errors raises the cost of doing independent research.

I don't have institutional backing. I'm not affiliated with Mozilla, Google, or any vendor. The goal of both papers is the same: improve the quality of publicly available evidence about mobile browser security. I made mistakes, I fixed them, I documented everything transparently. That's how research is supposed to work.

GrapheneOS's technical contributions to Android security are real and well-established. Disagreeing with specific claims in their advisory doesn't diminish that. And acknowledging my own errors doesn't mean the core argument was wrong – it means I'm willing to correct course when presented with better evidence.

0.7 6. Bottom Line

Here's what I got wrong and fixed: - `isolatedProcess` framing was imprecise (I called it "partially outdated" when it's fully accurate) - Left out Chromium's V8 sandbox, Oilpan, CFI, MTE, and other mitigations

- Didn't catch SpiderMonkey's missing CFI - Wrongly claimed hardware

mitigations are automatic - Poor CVE count framing without accounting for fuzzing investment - Implied Firefox extensions have better site isolation properties (they don't)

Here's what survived: - Firefox's Rust advantage is real and widening - Content blocking at the network layer is genuine pre-delivery prevention

- Chromium monoculture is a systemic risk - Browser choice is a

threat-model alignment, not binary "secure vs insecure" - The sandbox gap is Android-specific; on Windows, Firefox is at parity

The mistakes made the original paper weaker than it should have been. Fixing them makes the argument stronger, because now it accounts for the best counterarguments and still holds.

0.8 References

[1] Independent Security Research, "Comparative Analysis of Sandboxing and Mitigation Philosophies in Mobile User-Agent Architectures," Jul. 2026. [Online]. Available: https://spiritofstar.github.io/blog.html

[2] Security researchers, personal communication, Jul. 2026. Technical criticisms of [1] raised via public discussion.

[3] GrapheneOS, "Usage: Web Browsing." [Online]. Available: https://grapheneos.org/usage#web-browsing. Accessed: Jul. 2026.

[4] M. Geer et al., “CyberInsecurity: The Cost of Monopoly,” 2003. [Online]. Available: https://cryptome.org/cyberinsecurity.htm

[5] B. Schneier, “Schneier on Security,” various entries on security theater, security trade-offs, and adversarial thinking. [Online]. Available: https://www.schneier.com/

[6] Google Project Zero, “The State of 0-Day Mitigations,” 2021. [Online]. Available: https://googleprojectzero.blogspot.com/2021/02/debunking-myths-about-0-days.html

[7] Chromium Security, “V8 Sandbox,” Chromium Docs. [Online]. Available: https://chromium.googlesource.com/chromium/src/+/main/v8/src/sandbox/README.md

[8] Chromium Security, “Oilpan GC Design.” [Online]. Available: https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/platform/heap/BlinkGCDesign.md

[9] Mozilla, “Firefox Security Sandbox,” Mozilla Wiki. [Online]. Available: https://wiki.mozilla.org/Security/Sandbox. Accessed: Jul. 2026.

[10] Mozilla, “Integrating Project Fission (Site Isolation) in Firefox,” Mozilla Wiki. [Online]. Available: https://wiki.mozilla.org/Project_Fission. Accessed: Jul. 2026.

[11] M. Miller, “Site Isolation: A New Defense-in-Depth Security Architecture for the Web,” Chromium Blog, 2018. [Online]. Available: https://blog.chromium.org/2018/07/site-isolation-new-defense-in-depth.html

[12] Apple, “WebKit Sandboxing.” [Online]. Available: https://webkit.org/blog/14040/webkit-sandboxing/

[13] V. Nijim, “Building a More Secure Firefox with AppContainer,” Mozilla Security Blog, 2019. [Online]. Available: https://blog.mozilla.org/security/2019/05/22/building-a-more-secure-firefox-with-appcontainer/

[14] Chromium Project, “Memory Safety.” [Online]. Available: https://www.chromium.org/Home/chromium-security/memory-safety/. Accessed: Jul. 2026.

[15] Mozilla Security Blog, “Security Advisories.” [Online]. Available: https://www.mozilla.org/en-US/security/advisories/. Accessed: Jul. 2026.

[16] Chromium Project, “Control Flow Integrity,” Chromium Docs. [Online]. Available: https://chromium.googlesource.com/chromium/src/+/main/docs/security/control-flow-integrity.md. Accessed: Jul. 2026.

[17] Chromium, “PartitionAlloc Design,” Chromium Docs. [Online]. Available: https://chromium.googlesource.com/chromium/src/+/main/base/allocator/partition_allocator/PA_README.md. Accessed: Jul. 2026.

[18] Google, “OSS-Fuzz: Continuous Fuzzing for Open Source Software.” [Online]. Available: https://google.github.io/oss-fuzz/. Accessed: Jul. 2026.

[19] Chrome Developers, “Extension Runtime and Process Model.” [Online]. Available: https://developer.chrome.com/docs/extensions/mv3/process_model/. Accessed: Jul. 2026.